

Adaptive Batch Size and Learning Rate Scheduling for Efficient CNN Training

Tharun Swaminathan Ravi Kumar, Dev Gurung, and Jiong Zhang

Department of Computer Science and Engineering, University of North Texas, Denton, TX, USA

Abstract—Training convolutional neural networks requires choosing hyperparameters such as minibatch size and learning rate, yet the optimization behavior of a network changes throughout training. This work investigates whether dynamically adapting these parameters can improve the balance between classification performance and optimization effort. A custom CNN was trained on CIFAR-10 under four controlled configurations: fixed batch/fixed learning rate, adaptive batch/fixed learning rate, fixed batch/adaptive learning rate, and a combined adaptive strategy. The batch-size controller uses validation-loss behavior together with a three-epoch rolling measure of gradient-norm variability, while the learning-rate controller responds to validation behavior and batch-size changes. The fixed baseline achieved 56.60% test accuracy with 12,500 optimizer updates. Adaptive batch sizing reached 62.15% test accuracy with 10,316 updates, a 17.5% reduction. Adaptive learning rate alone reached 60.35%, while the combined strategy reached 61.70% with the same reduced update count. Under this frozen single-seed setting, adaptive batch sizing accounted for most of the observed improvement, showing that simple training-state signals can be used to improve the accuracy-efficiency tradeoff.

Keywords—*adaptive batch size, learning-rate scheduling, convolutional neural networks, CIFAR-10, gradient stability, deep learning optimization*

I. INTRODUCTION

Training a convolutional neural network depends strongly on optimization hyperparameters, particularly minibatch size and learning rate. These values are commonly selected before training and then held fixed or changed according to a predetermined schedule. That choice is convenient, but the optimization state of a network is not constant. Early epochs can contain highly variable gradients and rapid parameter changes, whereas later epochs can become more stable and require fewer aggressive updates.

This observation motivates a simple question: should batch size and learning rate remain fixed throughout training, or can they be adjusted automatically from signals already produced by the training process? Larger batches reduce the number of optimizer updates per epoch, but they also change gradient noise and can interact with the learning rate. Previous work has shown that the two quantities are closely related [1]–[4], while other studies have reported generalization concerns in large-batch regimes [5].

This project studies a lightweight controller-based approach on CIFAR-10 [7]. Validation behavior is used to determine whether progress has slowed, while minibatch gradient norms provide a practical indicator of optimization stability. Four controlled experiments isolate the effects of adapting batch size, learning rate, or both. The goal is not only to improve classification accuracy, but also to reduce optimization effort as measured by the number of parameter updates.

The project is deliberately framed as a controlled optimization study rather than a search for the highest possible CIFAR-10 accuracy. A compact network and fixed 20-epoch budget make it possible to observe controller decisions directly, compare update counts fairly, and keep the experiment reproducible on a single consumer GPU. The resulting analysis emphasizes how the training process changes when adaptation is enabled, not only the final score.

A. Problem Statement

Fixed optimization settings may be inefficient because the statistical behavior of the gradients changes throughout training. Small batches provide frequent updates but require more optimizer steps. Larger batches reduce update count but can alter optimization and generalization. Learning rate adds another interaction because an appropriate step size can depend on both the current training state and the batch size. The central question is therefore: can batch size and learning rate be adapted automatically using validation behavior and gradient stability while maintaining or improving predictive performance and reducing optimization effort?

B. Contributions

- 1) a grow-only adaptive batch controller that combines validation-loss plateau detection with a three-epoch rolling gradient-norm stability measure;
- 2) a validation-driven adaptive learning-rate controller with batch-dependent scaling;
- 3) a coupled strategy in which a selected batch change immediately influences the learning rate for the next epoch;
- 4) a four-condition ablation study under identical data, model, optimizer, seed, and epoch budget; and
- 5) a locked test protocol that separates controller tuning and checkpoint selection from final test evaluation.

II. RELATED WORK

Goyal et al. showed that large-minibatch ImageNet training can preserve accuracy when optimization difficulties are addressed with a linear learning-rate scaling rule and gradual warmup [1]. Their result established an important practical connection between batch size and learning rate. The present work follows the same broad motivation but differs in purpose and mechanism: it starts from batch size 32, changes batch size only when training behavior satisfies explicit conditions, uses square-root rather than linear scaling, and disables warmup in the frozen primary runs.

Smith et al. demonstrated that increasing batch size can, in many settings, replace learning-rate decay while requiring fewer parameter updates [2]. This idea is closely related to the E2 experiment in this study. The distinction is that E2 does not follow a predetermined growth schedule; the controller decides when to increase the batch size from validation and gradient behavior.

Balles et al. proposed adapting batch size from estimates of stochastic-gradient variance and explicitly coupling the batch size to the learning rate [3]. McCandlish et al. later introduced the gradient noise scale as an empirical predictor of the largest useful batch size and described the tradeoff between compute efficiency and time efficiency [4]. These studies support the idea that useful batch size changes during optimization. Our gradient-norm coefficient of variation is intentionally simpler: it is a stability indicator, not an estimator of gradient variance or formal gradient noise scale.

Large batches can also introduce generalization concerns. Keskar et al. reported a generalization gap in some large-batch settings and connected it experimentally with sharper minima [5]. For this reason, the controller used here increases batch size conservatively rather than beginning with the maximum allowed batch. Group Normalization is used throughout the model because its normalization statistics are independent of minibatch size, avoiding an additional batch-size-dependent effect in the comparison [6].

Together, these studies suggest that batch size, learning rate, and gradient noise should be viewed as interacting parts of the optimization process rather than independent knobs. Our contribution is intentionally modest: instead of reproducing any one large-batch recipe, we combine validation signals with a simple stability statistic and evaluate the resulting decisions in a transparent four-way ablation.

III. METHODOLOGY

A. Dataset and Preprocessing

CIFAR-10 contains 50,000 training images and 10,000 official test images across ten classes [7]. To keep the experiment practical on local hardware, a fixed stratified 25,000-image subset of the original training split was selected with seed 42 and divided into 20,000 training and 5,000 validation images. Each class contributed 2,000 training and 500 validation images. The complete 10,000-image official test split was kept untouched until all four primary configurations were frozen.

Training augmentation consisted of random 32x32 crops with four pixels of padding and random horizontal flips. Validation and test images were not augmented. All images were normalized with CIFAR-10 channel means (0.4914, 0.4822, 0.4465) and standard deviations (0.2470, 0.2435, 0.2616).

B. CNN Architecture and Optimization

All experiments used the same compact CNN: 3x3 convolutions with channel widths 32, 64, and 128; Group Normalization with eight groups; ReLU activations; max pooling after the 64- and 128-channel stages; adaptive global average pooling; and a linear 128-to-10 classifier. Group Normalization was chosen because its statistics do not depend on batch size [6]. Cross-entropy loss and SGD with momentum 0.9 and weight decay 5×10^{-4} were used. The common initial batch size was 32, the initial learning rate was 0.01, and every primary experiment used 20 epochs.

C. Gradient Stability Signal

For each minibatch, the global L2 norm of the gradients was computed over all trainable parameters. If θ_i denotes a parameter block and L is the loss, the global gradient norm was:

$$G = (\sum_i \|\nabla \theta_i L\|_2^2)^{1/2} \quad (1)$$

Within each epoch, the mean and population standard deviation of the minibatch gradient norms were recorded. Their coefficient of variation (CV) was used as an empirical measure of relative gradient variability:

$$CV = \sigma(G) / [\mu(G) + \varepsilon] \quad (2)$$

The controller used the mean of the most recent three epoch-level CV values. Lower values indicate less relative fluctuation in gradient magnitude. This statistic is used only as a practical stability signal and is not interpreted as a formal gradient-noise-scale estimator.

D. Adaptive Batch Controller

The batch controller used the grow-only sequence 32 -> 64 -> 128. A batch increase required both a validation-loss plateau and stable gradients. An improvement in validation loss had to exceed 0.01; after three epochs without such improvement, a plateau was declared. The rolling three-epoch gradient CV also had to be at or below 0.25. A two-epoch cooldown prevented immediate repeated growth. Decisions were made after validation, and the DataLoader was rebuilt so that the selected batch size was used beginning with the next epoch.

E. Adaptive Learning-Rate Controller

The learning-rate controller used reference batch size $B_{ref} = 32$ and reference learning rate $\eta_{ref} = 0.01$. When batch size changed, the base learning rate was recalculated using square-root scaling:

$$\eta(B) = \eta_{ref} (B / B_{ref})^{\alpha}, \quad \alpha = 0.5 \quad (3)$$

The controller also monitored validation loss and accuracy. The frozen configuration used a decay factor of 0.5, plateau patience of five epochs, worsening patience of three epochs, two cooldown epochs, minimum learning rate 10^{-5} , loss minimum delta 0.001, and accuracy minimum delta 0.002. Warmup was supported by the code but set to zero in the primary experiments. In E4, the batch decision was made first and the LR controller was then updated using the batch selected for the next epoch, creating a coupled strategy.

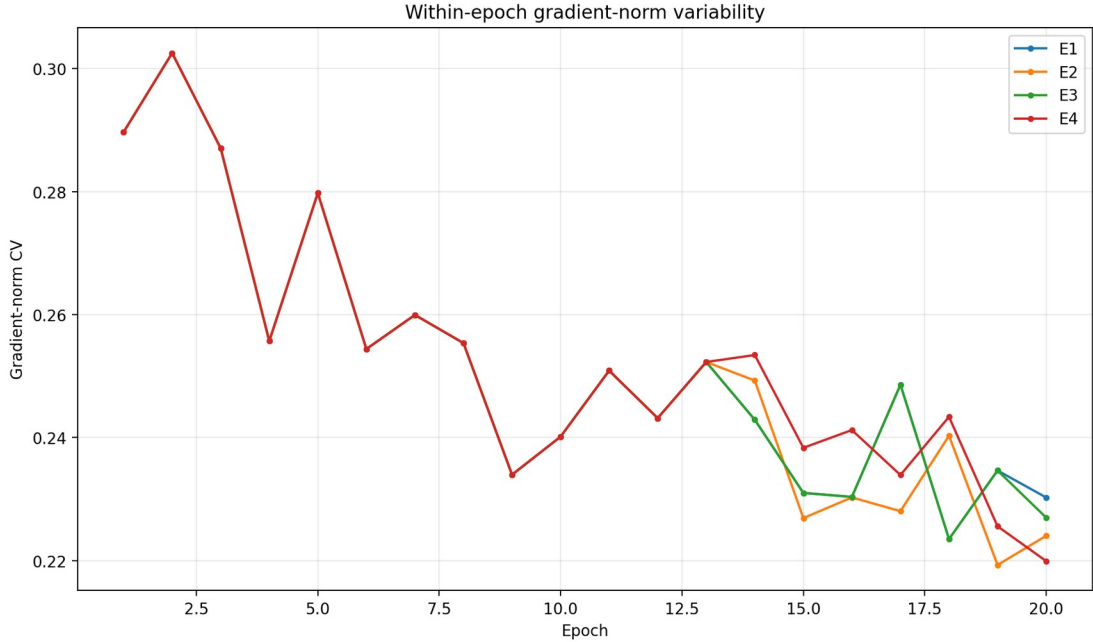


Fig. 1. Within-epoch gradient-norm CV across the primary runs; lower values indicate less relative variation.

IV. EXPERIMENTAL SETUP

TABLE I. PRIMARY EXPERIMENTAL DESIGN

Experiment	Batch Size	Learning Rate
E1	Fixed (32)	Fixed (0.01)
E2	Adaptive	Fixed (0.01)
E3	Fixed (32)	Adaptive
E4	Adaptive	Adaptive

The four conditions in Table I form a controlled ablation. E1 is the fixed baseline. E2 isolates adaptive batch sizing. E3 isolates adaptive learning rate. E4 enables both controllers. The same CNN, dataset split, seed 42, SGD optimizer, 20-epoch budget, augmentation, and Group Normalization were used throughout.

Primary runs were executed on an NVIDIA GeForce GTX 1660 Ti with 6 GB memory under Windows using Python 3.13, PyTorch 2.14.0 with CUDA 13.0 runtime support, two DataLoader workers, and pinned memory when CUDA was available. Python, NumPy, PyTorch, and CUDA random states were seeded, while deterministic cuDNN behavior was enabled and benchmarking disabled.

Checkpoint selection used validation loss only. The official test split was not used for controller tuning or model selection. After E1-E4 were frozen, a separate evaluator loaded the validation-selected checkpoint for each experiment and evaluated all four on the complete test set. Accuracy, cross-entropy loss, macro precision, macro recall, and macro F1 were

reported. Training efficiency was described using both optimizer updates and observed wall-clock time. Code and experiment scripts are maintained in the project repository: github.com/TharunSwaminathan/adaptive-batch-lr-cifar10.

V. RESULTS AND DISCUSSION

TABLE II. LOCKED VALIDATION AND TEST RESULTS

Exp.	Val. Acc.	Test Acc.	Test F1	Updates
E1	57.82%	56.60%	0.5577	12,500
E2	62.08%	62.15%	0.6089	10,316
E3	60.86%	60.35%	0.5964	12,500
E4	62.04%	61.70%	0.6020	10,316

A. Overall Performance

Table II summarizes the locked validation-selected checkpoints and final test results. The fixed baseline, E1, reached 56.60% test accuracy. E2 increased test accuracy to 62.15%, a gain of 5.55 percentage points, while reducing optimizer updates from 12,500 to 10,316. This corresponds to 2,184 fewer updates, or approximately 17.5%. E3 reached 60.35% test accuracy, 3.75 percentage points above E1. E4 reached 61.70% with the same 10,316 update count as E2.

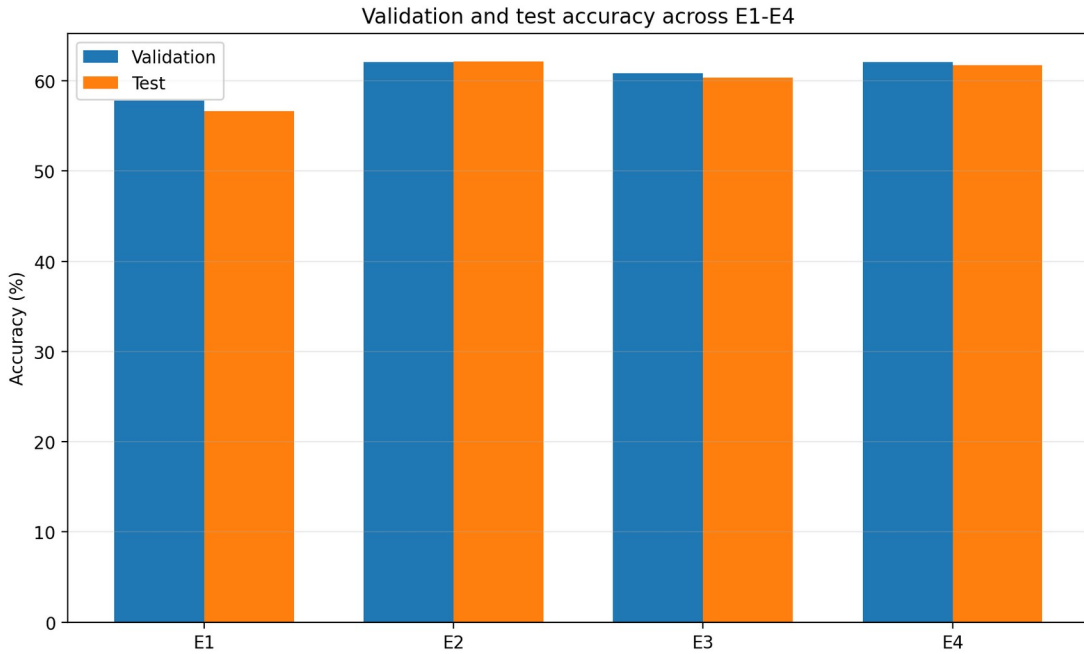


Fig. 2. Validation and test accuracy for the four primary experiments.

Validation and test accuracies remained close (Fig. 2). The validation-minus-test differences were +1.22 percentage points for E1, -0.07 for E2, +0.51 for E3, and +0.34 for E4. E2 and E4 differed by only 0.45 percentage points on the test set, which is too small to support a general superiority claim from one seed.

B. Training Dynamics and Controller Behavior

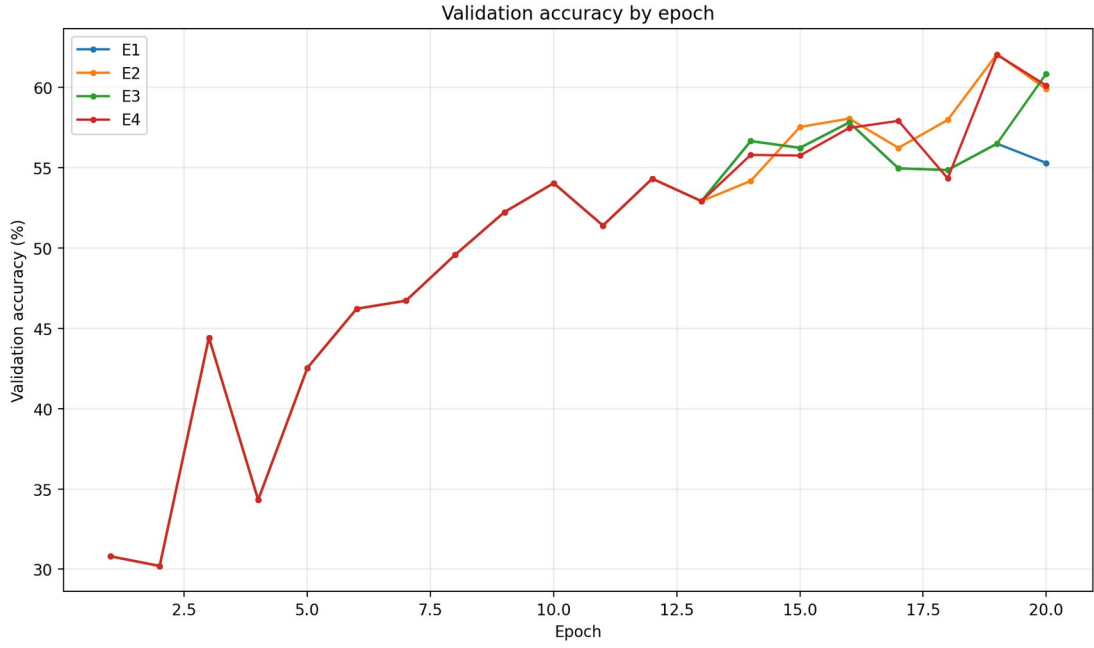


Fig. 3. Validation-accuracy trajectories across the 20-epoch budget.

The four runs were identical early in training and diverged only when an adaptive controller acted. E2 and E4 increased batch size after epoch 13, when the rolling gradient-CV stability score was 0.2488 and the validation plateau condition was satisfied. From epoch 14 onward, both used batch size 64; neither reached 128 within the remaining training budget.

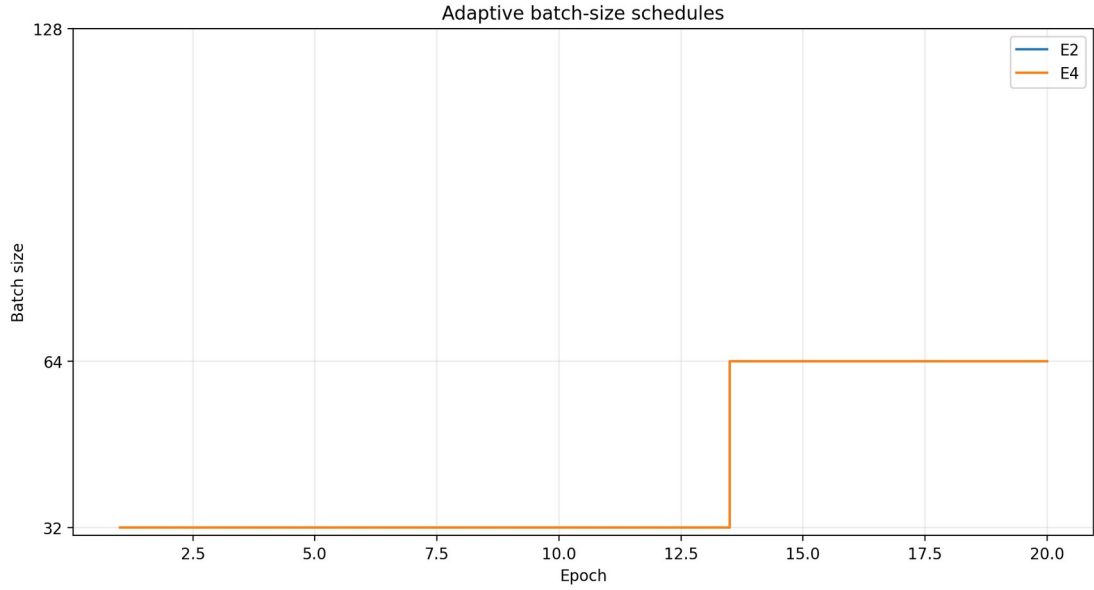


Fig. 4. Adaptive batch-size schedules for E2 and E4.

E3 kept batch size 32 and retained learning rate 0.01 through epoch 19. After sustained validation worsening, its controller selected 0.005, which was used in epoch 20. E4 behaved differently because its batch-size change immediately affected its learning rate: when batch size moved from 32 to 64, the square-root rule changed the learning rate to approximately 0.014142. That value remained active through the end of the run.

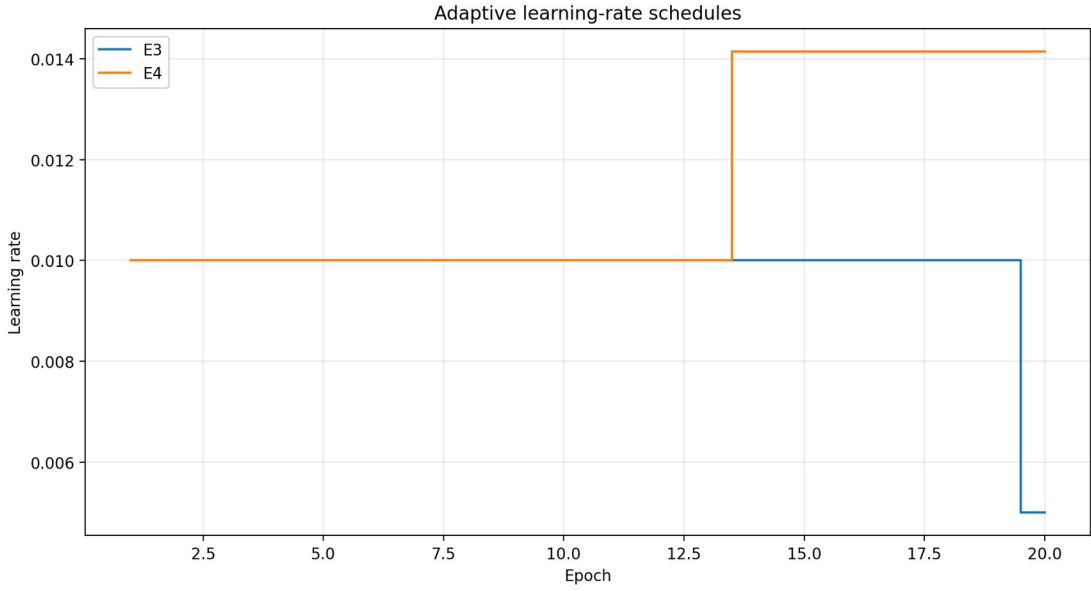


Fig. 5. Learning-rate schedules for the adaptive-LR experiments.

C. Optimization Effort and Interpretation

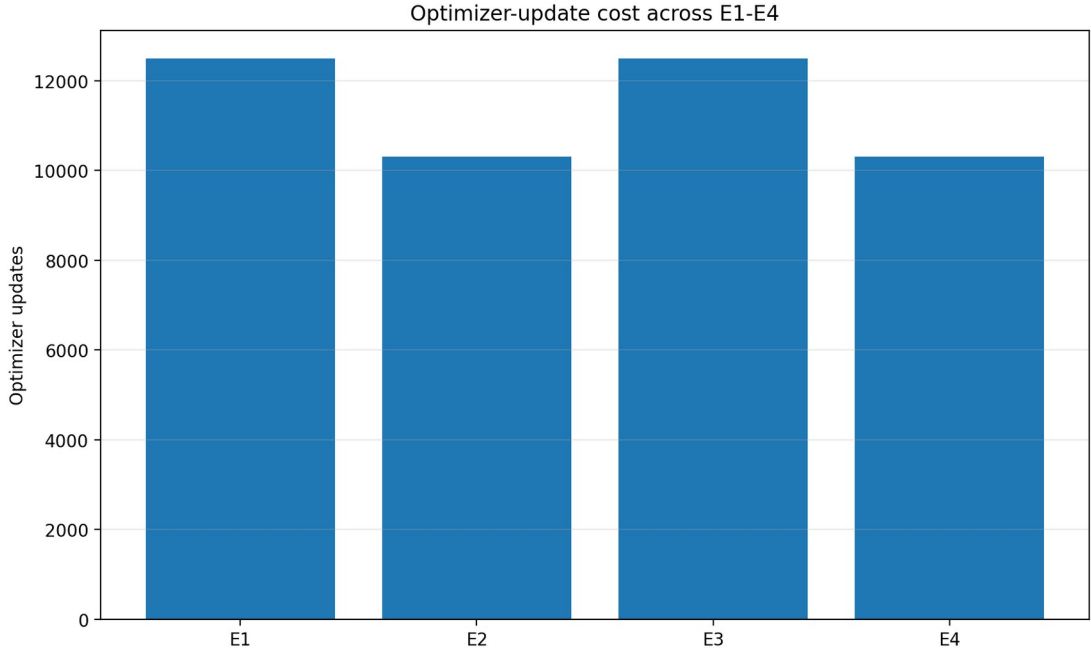


Fig. 6. Total optimizer updates for E1-E4.

The optimizer-update result provides the clearest efficiency difference. Both adaptive-batch experiments processed the same number of training examples per epoch but required 17.5% fewer parameter updates because later epochs used batch size 64. Wall-clock time was less consistent: E1, E3, and E4 required roughly 11 minutes, whereas the earlier E2 run took 1001.7 s despite fewer updates. Because runtime is affected by system load, data loading, and hardware state, the present study does not claim a wall-clock speedup from adaptive batching. Update count is the more stable computational-effort comparison.

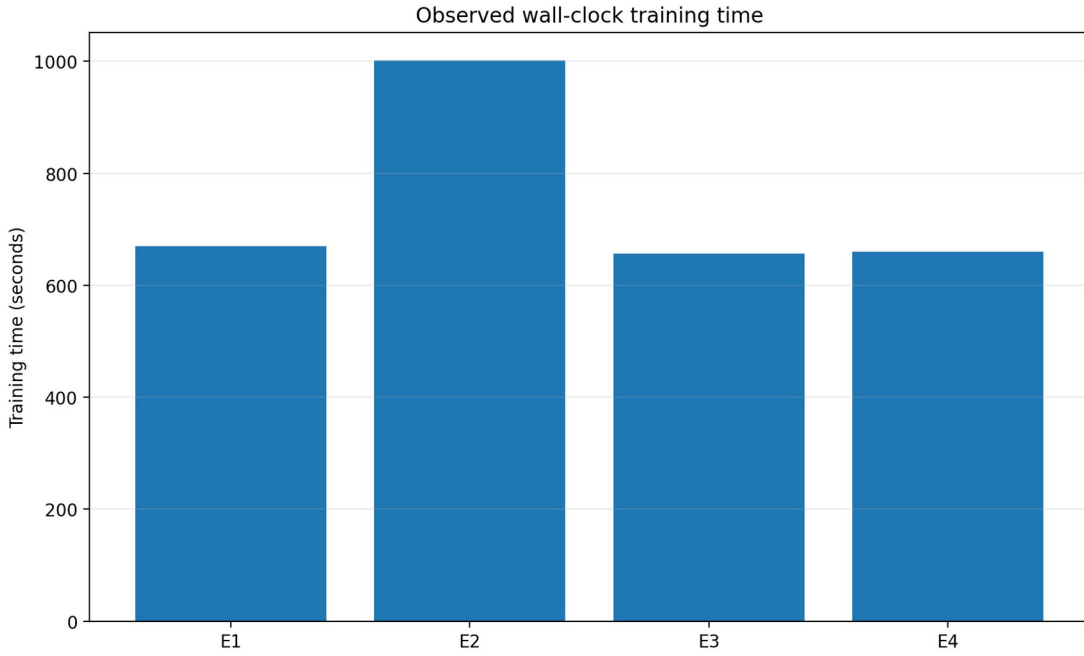


Fig. 7. Observed wall-clock training time. The E2 run is treated as hardware/runtime-specific rather than evidence of a systematic slowdown.

The main descriptive finding is that adaptive batch sizing accounted for most of the observed improvement under the frozen seed-42 setting. Adaptive learning rate also improved over the fixed baseline, but the combined controller did not provide an additional clear accuracy gain over E2. This outcome is consistent with the broader literature showing that batch size and learning rate interact [1]-[4], while also illustrating that coupling the two does not automatically guarantee a better final model.

VI. CONCLUSION AND FUTURE WORK

This project evaluated whether simple training-state signals could guide dynamic batch-size and learning-rate decisions in CNN training. The strongest observed improvement came from adaptive batch sizing: E2 raised test accuracy from 56.60% to 62.15% while reducing optimizer updates by 17.5%. Adaptive learning rate alone reached 60.35%, and the combined E4 strategy reached 61.70% with the same reduced update count as E2. The results therefore support adaptive control as a practical direction for improving the accuracy-efficiency tradeoff, while not establishing that the combined strategy is generally superior.

The main limitation is the single primary seed and the reduced 20,000-image training subset. Future work should repeat the frozen configurations across multiple seeds, use the full CIFAR-10 training set, and test deeper architectures such as ResNets. The gradient-norm CV could also be compared with formal gradient-variance or gradient-noise-scale estimators, and the grow-only batch controller could be extended to allow decreases when instability returns. Additional learning-rate policies, including linear scaling, warmup, cosine decay, or one-cycle schedules, would help clarify when batch and LR adaptation should act together. Finally, GPU utilization, memory use, throughput, and energy measurements would provide a broader view of hardware efficiency than optimizer updates alone.

REFERENCES

- [1] P. Goyal, P. Dollar, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” arXiv:1706.02677, 2017.
- [2] S. L. Smith, P.-J. Kindermans, C. Ying, and Q. V. Le, “Don’t Decay the Learning Rate, Increase the Batch Size,” in Proc. ICLR, 2018.
- [3] L. Balles, J. Romero, and P. Hennig, “Coupling Adaptive Batch Sizes with Learning Rates,” in Proc. UAI, 2017.
- [4] S. McCandlish, J. Kaplan, D. Amodei, and OpenAI Dota Team, “An Empirical Model of Large-Batch Training,” arXiv:1812.06162, 2018.
- [5] N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy, and P. T. P. Tang, “On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima,” in Proc. ICLR, 2017.
- [6] Y. Wu and K. He, “Group Normalization,” in Proc. European Conf. Computer Vision (ECCV), pp. 3-19, 2018.
- [7] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Univ. Toronto, Tech. Rep., 2009.